

wp-08-multiverse-superposition-inference

Multiverse Superposition Inference: Chaos-Seeded Adapter Ensembles with HeartScale Affinity Blending

Authors: Weslyn Whitehead Jr.¹

Affiliations:

¹ AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: <https://orcid.org/0009-0005-7707-3210>

Submitted: May 2026 · **Revision 2:** June 2026

Working Paper Series: AAMT-WP-08

Anchor DOI: 10.5281/zenodo.19600795

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Revision 2 changes: (C8) the per-adapter affinity now uses the resonance / product-Fisher–Rao kernel rather than raw inverse-L2, consistent with WP-15 §2.2 and Foundations Core Premise §2.3. (Framing) §6 makes precise the sense in which MSI is the tensor rank-lift of the rank-1 VortexGate routing (WP-01 §2.2/ §7.1), and notes that logit-space blending is a Product-of-Experts (weighted geometric mean) with a zero-veto property. No other content changed.

Abstract

Model ensemble methods typically combine multiple independently trained models by averaging logits, voting on outputs, or stacking predictions. For large language models, naive ensembling is prohibitive: N models requires $N \times \text{model_size}$ memory. We present **Multiverse Superposition Inference (MSI)**, a parameter-efficient ensemble method that trains N LoRA adapters on the same frozen base model using distinct chaos seeds in TERA space, then blends their logit outputs at inference time using HeartScale affinity scores as mixture weights. Because adapters share a base model, N -way ensembling requires only the base model in memory plus $N \times \text{adapter_size}$ (typically $N \times 1\text{--}5$ MB). The mixture weights are not learned — they are computed dynamically from the HeartScale coherence signature of the current prompt/context relative to each adapter’s training trajectory. The hypothesis is that TERA-space-diverse adapters (produced by different chaos seeds targeting different Meji regions during training) produce better-calibrated ensemble distributions than random-init-diverse or data-diverse adapters, because TERA-space diversity directly maps to semantic diversity

in the adapter’s learned representations. MSI is the tensor rank-lift of the single rank-1 VortexGate routing: where one gate is a single product (rank-1) Meji measure, an MSI ensemble realizes a rank- $\leq N$ mixture that can represent cross-dimension correlations a single gate provably cannot. MSI is specified as Tier-2 feature T2.3 of MaiaM Alchemist and exposed via the `alchemy.multiverse.predict` JSON-RPC method.

Keywords: ensemble methods, LoRA, mixture of adapters, uncertainty quantification, parameter-efficient fine-tuning, chaos seeding, logit blending, adapter diversity, tensor rank

1. Introduction

Series note: This paper is part of the AAMT Working Paper Series. The TERA projection algebra, 16-Meji routing system, and chaos seeding protocol are derived in WP-01 (Vortex-Gated MoE). The HeartScale affinity scoring used for blend weight computation is derived in WP-02 (HCRS). This paper uses both systems and provides only the definitions required locally.

Ensemble diversity is the key determinant of ensemble quality (Hansen & Salamon, 1990; Dietterich, 2000). Diverse models that make independent errors combine to produce a result superior to any individual member. The challenge for large language models is creating meaningful diversity at acceptable memory cost.

Standard diversity strategies for LLM ensembles include:

- **Random initialization diversity:** Train N models from N different random seeds. Error: this is computationally expensive (N full training runs) and does not guarantee semantic diversity.
- **Data diversity:** Partition the training data into N subsets; train one model per subset. Error: data partitioning reduces each model’s coverage.
- **Architecture diversity:** Use models of different sizes or architectures. Error: incompatible output spaces make blending non-trivial.

MSI introduces a fourth strategy: **TERA-space chaos-seed diversity**. Each adapter is trained from the same data with the same base model, but with a different chaos seed that biases training-time exploration toward a different region of the 16-Meji space. The resulting adapters have identical coverage of the training data but different representations of rare TERA regions — they make uncorrelated errors precisely in the regions where rare archetypes arise.

2. Training N Diverse Adapters

2.1 Chaos seed diversity

In standard training, the chaos seed protocol (WP-01 §4.4) selects seeds via `least_visited` or `farthest_point` from the training history. For MSI, each adapter $i \in \{0..N-1\}$ uses a biased seed schedule:

```
def msv_chaos_seed(adapter_index: int, n_adapters: int,
                  history: list[VortexProj]) -> TeraVector:
    """Seed biased toward adapter_index's assigned Meji partition."""
    partition_start = (adapter_index * 16) // n_adapters
    partition_end   = ((adapter_index + 1) * 16) // n_adapters
    partition_meji  = list(range(partition_start, partition_end))
    # Select least-visited Meji within this adapter's partition
    avg = np.mean([h.distribution for h in history], axis=0)
    k_star = min(partition_meji, key=lambda k: avg[k])
    return tera_from_meji_centroid(k_star)
```

For N=4, adapter 0 owns Meji 0–3 (red/grounding), adapter 1 owns Meji 4–7 (orange/creative), adapter 2 owns Meji 8–11 (yellow/systems), adapter 3 owns Meji 12–15 (green/generative). Each adapter develops specialized representations for its Meji partition while retaining general coverage of the full training set.

2.2 Training procedure

The outer loop trains N adapters sequentially (or in parallel where memory permits):

```
adapters = []
for i in range(config.n_parallel_seeds):
    run_config = base_config.copy()
    run_config.chaos.method = "msv_partition"
    run_config.chaos.msv_adapter_index = i
    run_config.chaos.msv_n_adapters = config.n_parallel_seeds
    adapter = train_single_adapter(run_config, base_model)
    adapters.append(adapter)
```

All N adapters share the same base model weights and the same training corpus. Only the chaos seed schedule differs.

3. HeartScale Affinity Blending

3.1 Per-adapter affinity scores

At inference time, the prompt is projected to a TERA coordinate. Each adapter's affinity to this TERA coordinate is computed from its historical mean TERA representation during training (stored in `adapter_config.json` as `training_tera_centroid`), using the resonance kernel (Foundations Core Premise §2.3) over the product-Fisher–Rao distance (WP-15 §2.2):

```
def compute_affinity(prompt_tera: TeraVector,
                    adapter: LoraAdapter) -> float:
    centroid = adapter.config.training_tera_centroid
    d = fisher_rao_distance(prompt_tera, centroid) #  $\sqrt{\sum_d (2 \cdot \arcsin \sqrt{u_d} - 2 \cdot \arcsin \sqrt{v_d})^2}$ 
```

```

phase = cos2_phase(prompt_tera, centroid)          #  $\cos^2(\Delta\phi)$ ; 1.0 when phase unavailable
# Resonance affinity:  $\cos^2(\Delta\phi) \cdot \exp(-d / \lambda)$ 
return phase * exp(-d / LAMBDA)                  # LAMBDA = resonance bandwidth

```

The resonance kernel $\cos^2(\Delta\phi) \cdot \exp(-d_{FR}/\lambda)$ replaces the raw inverse-distance $1/(\text{distance}+\epsilon)$ of Revision 1; it is bounded in $[0,1]$, smooth, and matches the affinity form used elsewhere in the AAMT framework. With $\cos^2(\Delta\phi)$ set to 1 (phase unavailable) it reduces to a Fisher–Rao RBF kernel.

3.2 Mixture weight computation

The N affinity scores are converted to mixture weights via temperature-scaled softmax:

```

def blend_weights(prompt_tera: TeraVector, adapters: list[LoraAdapter],
                  temperature: float = 0.5) -> list[float]:
    affinities = [compute_affinity(prompt_tera, a) for a in adapters]
    scaled = [a / temperature for a in affinities]
    weights = softmax(scaled)
    return weights

```

At $\text{temperature}=0.5$, the softmax is sharper than at $\text{temperature}=1.0$ — more weight concentrates on the closest adapter. At $\text{temperature}=0.1$ (winner-takes-all limit), only the closest adapter contributes.

3.3 Logit blending

```

def multiverse_predict(prompt: str, adapters: list[LoraAdapter],
                      base_model: BaseModel) -> torch.Tensor:
    prompt_tera = project_tera(embed(prompt))
    weights = blend_weights(prompt_tera, adapters)

    # Get logits from each adapter
    logit_sets = []
    for adapter, weight in zip(adapters, weights):
        merged_model = merge_lora(base_model, adapter)
        logits = forward(merged_model, prompt) # (vocab_size,)
        logit_sets.append(logits * weight)

    # Weighted sum in logit space
    blended_logits = sum(logit_sets)
    return blended_logits

```

Blending in logit space (rather than probability space) preserves the sharpness of individual adapter distributions while allowing the mixture to shift probability mass toward the regions where multiple adapters agree. **This is a Product-of-Experts:** $\text{softmax}(\sum_i w_i \cdot \text{logit}_i) \propto \prod_i p_i^{\{w_i\}}$, the weighted geometric mean of the adapter distributions. It therefore inherits a **zero-veto** property — if any single adapter assigns near-zero probability to a token, the blended probability is near zero — which is the inference-time form

of the multiplicative evaluation of Foundations Paper I and a useful safety property (one dissenting adapter can veto a token). An arithmetic (probability-space) mixture would instead realize the rank- $\leq N$ tensor mixture discussed in §6; both lift expressiveness beyond the single rank-1 gate.

4. Blend Strategy Comparison

Three blend strategies are available via `MultiverseConfig.blend_strategy` :

heartscale_softmax (default): Weights from HeartScale resonance affinity as described above. Adapts dynamically per prompt. Best for diverse prompts.

winner_takes_all : The adapter with highest affinity contributes all weight. Equivalent to routing — the ensemble collapses to a single expert. Useful for latency-critical settings.

uniform : Equal weights across all adapters. Acts as a standard ensemble average. Useful as a calibration baseline.

5. Memory Efficiency

For $N=4$ adapters at 4 MB each on a 5.2 GB base model:

Configuration	Memory
$N=4$ independent full models	$4 \times 5.2 \text{ GB} = 20.8 \text{ GB}$
$N=4$ LoRA adapters (MSI)	$5.2 \text{ GB} + 4 \times 4 \text{ MB} = 5.2 \text{ GB} + 16 \text{ MB}$
Overhead for ensembling	$\sim 0.3\%$

The base model is loaded once and shared. Adapter weights are CPU-resident and merged on demand. For $N=4$, this is a $>4\times$ memory reduction compared to naive ensembling.

6. Theoretical Motivation: TERA-Space Diversity as a Tensor Rank-Lift

Standard ensemble theory (Krogh & Vedelsby, 1995) decomposes ensemble error into bias and variance terms. Diversity — the average pairwise disagreement between ensemble members — reduces the variance term. For MSI, we claim that TERA-space diversity produces higher disagreement in the relevant regions (rare archetypes) than random initialization diversity.

Argument: Random initialization produces diversity that is uniformly distributed across the training data distribution. Rare archetypes, being rare, are also rare in the random-init diversity signal. TERA-chaos diversity, by contrast, is biased toward rare Meji regions by construction — each adapter specializes in the archetypes that

were under-represented in the other adapters' training. The ensemble disagreement is therefore concentrated precisely where it is most needed: in the tail of the TERA distribution.

Rank-lift formulation. A single VortexGate produces one product (rank-1) Meji measure (WP-01 §2.2), which by construction cannot represent correlations between TERA dimensions (WP-01 §7.1). An ensemble of N TERA-diverse adapters lifts this: an arithmetic mixture of N product measures is a nonnegative tensor of CP-rank $\leq N$, which *can* represent cross-dimension correlations; the logit-space (geometric) blend of §3.3 is the Product-of-Experts cousin of the same lift. This is the precise sense in which “superposition” is meant — a sum of product (separable) states is, in general, an entangled (non-separable) state. The free parameter N trades expressiveness for cost; Foundations Paper III (quadratic scaling, $V(N) = \alpha N - \beta N^2$) bounds the useful N at the value peak $N^* = \alpha / (2\beta)$.

This claim is empirically testable by comparing ensemble calibration curves across the four blend strategies on prompts stratified by Odu cell frequency.

7. Relationship to Prior Work

LoRA ensembles (He et al., 2023): Train multiple LoRA adapters and average their outputs. Does not address diversity; uses uniform blending.

AdapterSoup (Chronopoulou et al., 2023): Average LoRA adapter weights (not logits) from adapters trained on different tasks. MSI blends logits, not weights, which avoids weight-space interference between adapters trained on different TERA regions.

MoE at the adapter level (Zadouri et al., 2023): Route inputs to one of N task-specific adapters. MSI blends N adapters rather than routing to one.

Key distinction: MSI is the first adapter ensemble method where diversity is deliberately induced by symbolic TERA-space seeding rather than random initialization or data partitioning, where mixture weights are computed from a geometry-based resonance affinity rather than being learned or uniform, and where the construction is understood as a tensor rank-lift of a rank-1 symbolic router.

8. Conclusion

MSI provides a parameter-efficient ensemble method for LoRA-based MoE models where diversity is a first-class design artifact rather than a byproduct of random initialization. TERA-chaos seeding produces adapters that specialize in different regions of the archetypal space; HeartScale resonance-affinity blending routes inference-time mixing toward the most contextually relevant adapter combination. The memory overhead is negligible ($N \times \text{adapter_size}$ on top of a single base model), the blend strategy is configurable from winner-takes-all to soft uniform mixture, and the method has a clean interpretation as the rank-lift of the rank-1

VortexGate routing. MSI represents a practical path toward ensemble-quality inference at single-model memory cost.

Acknowledgments

The author thanks the AsAManThinks Research community.

Funding

Self-funded through AsAManThinks Research.

Data Availability

The MSI training and inference specification is described herein. Implementation is in MaiaM Alchemist T2.3 (`multiverse/adaptor_bank.py` , `multiverse/heartscale_blender.py`).

Conflict of Interest

The author is the founder and CEO of AsAManThinks Research.

References

- Chronopoulou, A., Peters, M. E., Fraser, A., & Dodge, J. (2023). AdapterSoup: Weight averaging to improve generalization of pretrained language models. In *Findings of EACL 2023* (pp. 2054–2063). <https://doi.org/10.18653/v1/2023.findings-eacl.153>
- Dietterich, T. G. (2000). Ensemble methods in machine learning. In *Proceedings of MCS 2000* (pp. 1–15). https://doi.org/10.1007/3-540-45014-9_1
- Hansen, L. K., & Salamon, P. (1990). Neural network ensembles. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(10), 993–1001. <https://doi.org/10.1109/34.58871>
- He, J., Zhou, C., Ma, X., Berg-Kirkpatrick, T., & Neubig, G. (2023). Towards a unified view of parameter-efficient transfer learning. In *Proceedings of ICLR 2022*. <https://doi.org/10.48550/arXiv.2110.04366>
- Krogh, A., & Vedelsby, J. (1995). Neural network ensembles, cross validation, and active learning. In *Advances in Neural Information Processing Systems 7* (pp. 231–238).
- Zadouri, T., Üstün, A., Ahmadian, A., Ermiş, B., Zettlemoyer, L., & Hooker, S. (2023). Pushing mixture of experts to the limit: Extremely parameter efficient MoE for instruction tuning. *arXiv preprint*. <https://doi.org/10.48550/arXiv.2309.05444>

Whitehead, W. (2026). MaiaM Wings: Consciousness-Aligned Language Architecture. *Zenodo*.
<https://doi.org/10.5281/zenodo.19600795>